

National University of Computer and Emerging Sciences
Islamabad Campus

**Object-Oriented Programming
(CS 1004)**

Course Instructor(s):
Mr. Pir Sami Ullah Shah, Ms. Mahnoor Ayaz

Section(s): SE (A, B, C, D)

Final Exam

Total Time (Hrs): 3

Total Marks: 125

Total Questions: 2

Date: May 18, 2026

Roll No	Course Section	Student Signature
---------	----------------	-------------------

Important Information: The use or possession of mobile phones and other electronic devices during the exam is strictly prohibited, whether the device is powered on or off. Do not bring mobile phones, smartwatches, tablets, or other electronic gadgets into the examination hall. If you are found with any prohibited devices, your exam may be cancelled, and further disciplinary action may be taken.

Do not write below this line.

Attempt all the questions.

[CLO 2: Applying OOP Concepts to computing problems for the related program]

Q1: Analyze each provided code snippet to detect and correct any compilation or runtime errors, then determine the exact output while assume all necessary header files (`#include`) are present.

[15 * 5 = 75 marks]

a)	<pre>void alloc(int*** ppp) { *ppp = new int*[2]; (*ppp)[0] = new int(7); (*ppp)[1] = new int(9); } int main() { int** pp = nullptr; alloc(&pp); cout << **pp << " " << *pp[1] << " " << *(pp+1); delete pp[0]; delete pp[1]; delete[] pp; }</pre>	Output 7 9 9
b)	<pre>int moves = 0; void hanoi(int n, char a, char b, char c) { if (n == 0) return; hanoi(n-1, a, c, b);</pre>	Output 15

National University of Computer and Emerging Sciences
Islamabad Campus

	<pre> moves++; hanoi(n-1, b, a, c); } int main() { hanoi(4, 'A', 'B', 'C'); cout << moves; } </pre>	
c)	<pre> struct P { int a, b; }; P arr[3] = {{1,2},{3,4},{5,6}}; P* p = arr; cout << (p+2)->a << " " << p[1].b << " " << (++p)->a; </pre>	Output 5 4 3
d)	<pre> class V { protected: int value; public: V() : value(1) { cout << "V "; } virtual int compute() const { return value * 2; } }; class A : virtual public V { public: A() { value += 2; cout << "A "; } }; class B : virtual public V { public: B() { value *= 3; cout << "B "; } }; class C : public A, public B { public: C() { value += compute(); cout << "C: " << value << endl; } }; int main() { C obj; return 0; } </pre>	Output V A B C: 27
e)	<pre> class Base { public: int a = 10; virtual int get() const { return a + 5; } virtual void show() const { cout << "Base: " << get() << endl; } }; class Derived : public Base { public: </pre>	Output Base: 15 Derived: 40

National University of Computer and Emerging Sciences

Islamabad Campus

	<pre> int a = 20; int get() const override { return a * 2; } void show() const override { cout << "Derived: " << get() << endl; } }; void process(Base b) { b.show(); } void processRef(Base& b) { b.show(); } int main() { Derived d; process(d); processRef(d); return 0; } </pre>	
f)	<pre> class A { public: virtual int calc() { return 1; } virtual void show() { cout << "A: " << calc() << endl; } }; class B : public A { public: int calc() override { return A::calc() + 2; } void show() override { int val = calc() * 3; cout << "B: " << val << endl; } }; class C : public B { public: int calc() override { int base = B::calc(); return base * base; } }; int main() { A* a = new C(); a->show(); return 0; } </pre>	Output B: 27

National University of Computer and Emerging Sciences
Islamabad Campus

g)	<pre> class Engine { public: virtual int power() const { return 100; } }; class Turbo : public Engine { public: int power() const override { return 200; } }; void test(const Engine& e) { auto func = [&]() { cout << e.power() << endl; }; func(); } int main() { Turbo t; test(t); return 0; } </pre>	200
h)	<pre> class Base { public: virtual Base* clone() const { cout << "Base clone\n"; return new Base(*this); } }; class Derived : public Base { public: Derived* clone() const override { cout << "Derived clone\n"; return new Derived(*this); } }; void cloneTest(const Base& obj) { Base* copy = obj.clone(); delete copy; } int main() { Derived d; cloneTest(d); return 0;} </pre>	Output Derived clone

National University of Computer and Emerging Sciences

Islamabad Campus

i)	<pre> class Base { public: Base(int x) { cout << "Base: " << x << endl; } }; class Left : virtual public Base { public: Left(int x) : Base(x) { cout << "Left\n"; } }; class Right : virtual public Base { public: Right(int x) : Base(x + 1) { cout << "Right\n"; } }; class Final : public Left, public Right { public: Final() : Base(3), Left(0), Right(0) { cout << "Final\n"; } }; int main() { Final f; return 0;} </pre>	Base: 3 Left Right Final
j)	<pre> int f(int n) { static int sum = 0; if (n == 0) return sum; sum += n; return f(n - 1); } int main() { cout << f(4) << " " << f(3); } </pre>	Output 10 16
k)	<pre> template<typename T, typename U> class P { public: void f(){cout<<"1 ";} }; template<typename T> class P<T, T> { public: void f(){cout<<"2 ";} }; </pre>	Output 1 2 3 4

National University of Computer and Emerging Sciences
Islamabad Campus

	<pre> template<typename T> class P<T*, T*> { public: void f(){cout<<"3 ";} }; template<> class P<int*, int*> { public: void f(){cout<<"4 ";} }; int main() { P<int, double> a; a.f(); P<int, int> b; b.f(); P<char*, char*> c; c.f(); P<int*, int*> d; d.f(); } </pre>	
l)	<pre> class Logger { public: Logger(){cout<<"L ";} virtual ~Logger(){cout<<"~L ";} }; class FileLogger : public Logger { public: FileLogger(){cout<<"FL ";} ~FileLogger(){cout<<"~FL ";} }; class Service { Logger* lg; public: Service(): lg(new FileLogger()) { cout << "S "; } ~Service() { delete lg; cout << "~S "; } }; int main() { Service s; } </pre>	Output L FL S ~FL ~L ~S
m)	<pre> class Sensor { public: int reading = 5; Sensor(){cout<<"S ";} ~Sensor(){cout<<"~S ";} </pre>	Output S M 99 ~M ~S

National University of Computer and Emerging Sciences
Islamabad Campus

	<pre> }; class Monitor { Sensor& s; public: Monitor(Sensor& sn): s(sn){cout<<"M ";} void check(){ cout << s.reading << " "; } ~Monitor(){cout<<"~M ";} }; int main() { Sensor sn; Monitor m(sn); sn.reading = 99; m.check(); } </pre>	
n)	<pre> class Entity { protected: int id; static int counter; public: Entity() : id(++counter) { cout << "Entity#" << id << endl; } Entity(const Entity& o) : id(++counter) { cout << "Entity#" << id << " copied from #" << o.id << endl; } virtual void identify() const { cout << "Entity#" << id << endl; } virtual Entity* clone() const { return new Entity(*this); } virtual ~Entity() { cout << "~Entity#" << id << endl; } }; int Entity::counter = 0; class Agent : virtual public Entity { protected: string role; public: Agent(string r = "none") : role(r) { cout << "Agent role=" << role << endl; } Agent(const Agent& o) : Entity(o), role(o.role + "_copy") { cout << "Agent copied role=" << role << endl; } void identify() const { cout << "Agent#" << id << " role=" << role << endl; } Agent* clone() const { return new Agent(*this); } ~Agent() { cout << "~Agent role=" << role << endl; }; } </pre>	Output Entity#1 Agent role=CEO Supervisor level=5 Manager Manager#1 role=CEO level=5 Manager#1 role=CEO level=5 Entity#2 Agent role=Dev Entity#3 copied from #2 Agent copied role=Dev_copy Agent#3 role=Dev_copy --- cleanup --- ~Agent role=Dev ~Entity#2 ~Agent role=Dev_copy ~Entity#3 ~Manager ~Supervisor level=5 ~Agent role=CEO ~Entity#1

National University of Computer and Emerging Sciences

Islamabad Campus

	<pre> class Supervisor : virtual public Entity { protected: int level; public: Supervisor(int l = 1) : level(l) { cout << "Supervisor level=" << level << endl; } void identify() const { cout << "Supervisor#" << id << " level=" << level << endl; } ~Supervisor() { cout << "~Supervisor level=" << level << endl; } }; class Manager : public Agent, public Supervisor { public: Manager(string r, int l) : Entity(), Agent(r), Supervisor(l) { cout << "Manager" << endl; } void identify() const { cout << "Manager#" << id << " role=" << role << " level=" << level << endl; } ~Manager() { cout << "~Manager" << endl; } }; int main() { Manager m("CEO", 5); m.identify(); Entity* e = &m; e->identify(); Agent* a = new Agent("Dev"); Entity* ec = a->clone(); ec->identify(); cout << "--- cleanup ---" << endl; delete a; delete ec; } </pre>	
o)	<pre> template<typename T> void f(T x) { cout << "val "; } template<typename T> void f(T& x) { cout << "ref "; } template<typename T> void f(const T& x){ cout << "cref "; } int main() { int a = 5; const int b = 5; f(a); f(int>(a); f(b); } </pre>	Output Error

National University of Computer and Emerging Sciences

Islamabad Campus

[CLO 3: Model an algorithm solution using OOP concepts for the given problem]

Q2: Review the case study to understand its logic, fill in all missing code segments, and ensure the completed program generates the exact output given below.

You are building MediTrack, a hospital management system used by NovaSphere's HealthTech client. The system must manage doctors, patients, departments, and medical records. It uses inheritance for staff hierarchy, templates for generic record storage, polymorphism for billing, composition for departments owning equipment, and aggregation for hospitals referencing doctors who can move between facilities.

You will be given a partially completed program with 10 blanks (**BLANK 1**, **BLANK 2**, etc.). Your job: fill each blank so the program compiles and produces the exact expected output at the bottom. Lastly, provide each **BLANKED CODE** in the provided table given on the last page of the paper.

Code:

```
#include <iostream>
#include <string>
#include <cstring>
using namespace std;

template<typename T, int N>
class Registry {
    T data[N];
    int count;
public:
    Registry() : count(0) { cout << "Registry["<<N<<"] "; }

    bool add(BLANK 1) {
        if (count >= N) return false;
        data[count++] = item;
        return true;
    }

    T& get(int i) { return data[i]; }
    int size() const { return count; }
};
```

National University of Computer and Emerging Sciences
Islamabad Campus

```
struct Equipment {
    string name;
    int id;
    Equipment(string n = "", int i = 0) : name(n), id(i) {
        cout << "Eq("<<name<<") ";
    }
    ~Equipment() { cout << "~Eq("<<name<<") "; }
};

class Billable {
public:
    virtual double computeBill() {BLANK 2}
    virtual void describe() { cout << "Billable "; }
    virtual ~Billable() { cout << "~Billable "; }
};

class Person {
protected:
    string name;
    int age;
public:
    Person(string n, int a) : name(n), age(a) { cout << "P("<<name<<") "; }
    virtual ~Person() { cout << "~P("<<name<<") "; }
    virtual void show() { cout << "Person:"<<name<< " "; }
};

class Staff : public Person {
protected:
    int staffId;
    static int totalStaff;
public:
    Staff(string n, int a, int id) BLANK 3
    {
        totalStaff++;
        cout << "S("<<staffId<<") ";
    }
    ~Staff() { cout << "~S("<<staffId<<") "; }
    static int getTotal() { return totalStaff; }
};

BLANK 4

class Doctor : public Staff, public Billable {
    double consultFee;
public:
```

National University of Computer and Emerging Sciences

Islamabad Campus

```
Doctor(string n, int a, int id, double fee)
: Staff(n, a, id), consultFee(fee) {
    cout << "D("<<name<<") ";
}
~Doctor() { cout << "~D("<<name<<") "; }

double computeBill() BLANK 5 {
    return consultFee;
}
void describe() override { cout << "Dr."<<name<< " "; }
};

class Patient : public Person, public Billable {
    int visits;
    double dailyRate;
public:
    Patient(string n, int a, int v, double rate)
        : Person(n, a), visits(v), dailyRate(rate) {
        cout << "Pt("<<name<<") ";
    }
    ~Patient() { cout << "~Pt("<<name<<") "; }

    double computeBill() override {
        return billRec(visits);
    }
private:
    double billRec(int v) {
        if (v <= 0) return 0;
        BLANK 6
        return dailyRate + 0.5 * billRec(v - 1);
    }
    void describe() override { cout << "Pt:"<<name<< " "; }
};

class Department {
    string name;
    Equipment primary;
    Equipment secondary;
public:
    Department(string n, string e1, int id1, string e2, int id2)
        : name(n), primary(e1, id1), secondary(e2, id2) {
        cout << "Dept("<<name<<") ";
    }
    ~Department() { cout << "~Dept("<<name<<") "; }
};
```

National University of Computer and Emerging Sciences

Islamabad Campus

```
class Hospital {
    string name;
    Doctor** doctors;
    int docCount;
    Department* dept;
public:
    Hospital(string n, Doctor** docs, int count, Department* d)
        : name(n), doctors(docs), docCount(count), dept(d) {
        cout << "H("<<name<<") ";
    }
    ~Hospital() {
        BLANK 7
        delete dept;
        cout << "~H("<<name<<") ";
    }

    double totalRevenue() {
        double sum = 0;
        for (int i = 0; i < docCount; i++)
            sum += doctors[i]->computeBill();
        return sum;
    }

    void diagnose(Billable* b) {
        b->describe();
        cout << "bill="<<b->computeBill()<<" ";
    }
};

int main() {
    cout << "[INIT] ";

    Department* cardio = new Department("Cardio", "ECG", 101, "MRI", 102);
    cout << "| ";

    Doctor* d1 = new Doctor("Omer", 40, 1, 150.0);
    Doctor* d2 = new Doctor("Sami", 35, 2, 200.0);
    cout << "| ";

    Doctor* docs[2] = { d1, d2 };

    Hospital* h = new Hospital("NovaCare", docs, 2, cardio);
    cout << "| ";

    Patient* pt = new Patient("Ali", 30, 3, 100.0);
```

National University of Computer and Emerging Sciences

Islamabad Campus

```
cout << "| ";

Billable* items[3] = { d1, d2, pt };
cout << "[BILLS] ";
for (int i = 0; i < 3; i++) {
    BLANK 8
    items[i]->describe();
    cout << items[i]->computeBill() << " ";
}

cout << "| ";
Registry<BLANK 9> reg;
reg.add(d1);
reg.add(d2);
cout << "regSize=" << reg.size() << " ";

cout << "staff=" << BLANK 10 << " ";

cout << "rev=" << h->totalRevenue() << " ";

cout << "ptBill=" << pt->computeBill() << " ";

cout << "| [CLEAN] ";
delete pt;
delete h;
delete d1;
delete d2;

cout << "[DONE]";
return 0;
}
```

Output

```
[INIT] Eq(ECG) Eq(MRI) Dept(Cardio) | P(Omer) S(1) D(Omer) P(Sami) S(2) D(Sami) | H(NovaCare) |
P(Ali) Pt(Ali) | [BILLS] Dr.Omer 150 Dr.Sami 200 Pt:Ali 175 | Registry[5] regSize=2 staff=2 rev=350
ptBill=175 | [CLEAN] ~Pt(Ali) ~Billable ~P(Ali) ~Dept(Cardio) ~Eq(MRI) ~Eq(ECG) ~H(NovaCare)
~D(Omer) ~Billable ~S(1) ~P(Omer) ~D(Sami) ~Billable ~S(2) ~P(Sami) [DONE]
```

National University of Computer and Emerging Sciences

Islamabad Campus

Note: Insert all missing code blocks exclusively within the provided tabular format.

[10 * 5 = 50 marks]

Blank 1	Blank	Code
Blank 2 1		<code>const T& item</code>
Blank 3 2		<code>return 0.0;</code>
Blank 4 3		<code>: Person(n, a), staffId(id)</code>
Blank 5 4		<code>int Staff::totalStaff = 0;</code>
Blank 6 5		<code>override</code>
Blank 7 6		<code>if (v == 1) return dailyRate;</code>
Blank 8 7		<code>delete[] doctors;</code>
Blank 9 8		<code>h->diagnose(items[i]);</code>
Blank 10 9		<code>Doctor*, 5</code>
	10	<code>Staff::getTotal()</code>

The End.